I think you'll agree that the handler parameters for this method are somewhat hard to read. The parameters define the expected shape of the request, which isn't ideal. The `[AsParameters]` attribute lets you wrap all these arguments into a single `class` or `struct`, simplifying the method signature and making everything more readable.

Listing 7.9 shows an example of converting this endpoint to use `[AsParameters]` by replacing it with a `record struct`. You could also use a `class`, `record`, or `struct`, and you can use properties instead of constructor parameters if you prefer. See the documentation for all the permutations available at http://mng.bz/a1KB.

Listing 7.9 Using `[AsParameters]` to simplify endpoint handler parameters

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/category/{id}",
    ([AsParameters] SearchModel model) => $"Received {model}");  ❶

app.Run();

record struct SearchModel(
    int id,                                                      ❷
    int page,                                                    ❷
    [FromHeader(Name = "sort")] bool? sortAsc,                   ❷
    [FromQuery(Name = "q")] string search);                      ❷
```

❶ [AsParameters] indicates that the constructor or properties of the type should be bound, not the type itself.

❷ Each parameter is bound as though it were written in the endpoint handler.

The same attributes and rules apply for binding an `[AsParameters]` type's constructor parameters and binding endpoint handler parameters, so you can use `[From*]` attributes, inject services and well-known types,